Comparisons of different AI techniques on the game 2048

Written by Denis Legue (000513162), Robin Marchand, Gabriel, Monwar Mahtab Hawlader (000514679)

ULB
INFO-H410

Abstract

GitHub Repository —
https://github.com/Somsro/infoH410

Introduction

In the course INFO-H410 Techniques of Artificial Intelligence at ULB, students are required to apply and compare different AI techniques to the same problem. The game chosen is 2048. 2048 is a single-player game on a 4×4 grid where the player has to combine tiles by shifting them in one of the four possible directions: up, down, left and right. Each turn, a new tile with a value of 2 or 4 appears in an empty spot on the board. The player wins by creating a tile with the value of 2048, but can continue playing to reach higher values. The game ends when there are no more valid moves available.

The randomness of the games makes it a good candidate for testing the performance of different AI techniques. Three agents will be compared; Deep qleaner, Expectimax and an n-tuple network TD learning agent. The goal will be to evaluate their performance (the final score reached) but also the computational cost to better understand the trade-offs between planning and leanred polcies.

Problem Formulation

The game 2048 is a sequential decision making problem with uncertainty. The state of the game is represented by the configuration of the board. The configuration of the board is a 4×4 grid where each cell can be empty or contain a tile with a value that is a power of 2. At each time step, the agent observes the current board configuration and selects in which direction to shift the tiles in one of the four possible directions: up, down, left and right as long as there is at least one tile moving. Two tiles with the same value that collide during a shift will merge into a tile with the value of their sum. A new tile with a value of 2 or 4 will then appear in a random empty spot on the board which makes future states only partially controllable by the agent. The objective of the agents is to achieve the highest score. The game ends when there are no more valid moves available. The fact that a new tile appears in a random empty spot on the board after each move makes the game stochastic and thus making it a good candidate for testing the performance of different AI techniques.

Formal Representation:

State Space (S): Values and grid positions of all tiles.
Actions (A): $\{Up, Down, Left, Right\}$, restricted to moves that modify the board.
Reward (r): Can be constructed based on several features (empty cells, monotonicity, and max tile placement).
Constraints: Terminal state is reached when no legal actions remain. Tabular Q-learning and exhaustive search are infeasible due to the massive state space.
Probabilities: Stochastic transitions $P(s' \mid s, a)$ dictate the game dynamics, preventing exact deterministic planning.

Methodologies

Three agents will be compared; Deep Q-Learning, Expectimax and an n-tuple network TD(0) learning agent. The three methods have each their own field of application and are based on different principles. Expectimax is a planning-based approach, deep Q-Learning is a deep reinforcement learning method that learns action values from experience meanwhile TD learning with function approximation is a lighter value-based alternative. Comparing them on the same problem will allow us to better understand the trade-offs between planning, learning, the computational cost as well as performance of each method. For each of the following approach, the states is encoded as a list of 16 integers, each representing the exponant k such that the tile value in that cell is 2^k. (For example, 16 is encoded as 4 since $2^4 = 16$ with the particularity that 0 represents an empty cell). The action space is the same for all agents and consists of the 4 possible moves: up, down, left and right encoded as integer values from 0 to 3.

Approach: Expectimax

Approach: Deep Q-Learning (DQN)

Logic: Approximate the action value function (Q-value) using a deep neural network to bypass the exhaustive explo-

ration of the state space. The agent learns directly from experience via stochastic gradient descent on (s, a, r, s') tuples, guided by a shaped reward system reflecting 2048 heuristics (tile merges, empty cells, monotonicity, and corner penalties).

Assumption: The environment features a massive state space and stochastic transitions, rendering tabular methods infeasible. While this model-free approach is good at capturing complex patterns, it assumes access to high computational power and massive amounts of training data (very long training sessions) to converge.

Approach: N-tuple network TD(0) learning agent

Temporal-Difference learning with value function approximation provides a lighter alternative to Deep Q-Learning. Instead of learning action values directly, this approach estimates the value of board after-states and updates this estimate by bootstrapping from successor after-states. In this implementation, the after-states represent the move without placing a new tile yet in order to estimate the value function. An error signal is computed based on the difference between the current value estimate and the observed reward plus the value of the next after-state. The value function is approximated using an N-tuple network composed of lookup tables defined over several tile patterns and their symmetric variants.

Algorithm 1: Training procedure of the TD agent

Require: Number of episodes N
Initialize TD agent with N-tuple network $V\theta$
for episode $= 1$ to N **do**
    Reset the environment
    Initialize empty trajectory path
    done $\leftarrow$ false
    while not done **do**
        Select an action using ϵ-greedy exploration
        Clone the environment and apply the action to obtain the after-state and reward
        Execute the action in the real environment
        Store the after-state, reward, and terminal flag in the path
    end while
    $target \leftarrow 0$
    for each stored after-state s in reverse order **do**
        $v \leftarrow V\theta(s)$
        $\delta \leftarrow target - v$
        Update the weights of $V\theta$ using δ
        Update $target \leftarrow r + V\theta(s)$
    end for
    Update $\epsilon \leftarrow \max(\epsilon_{\min}, \epsilon \cdot \lambda)$
end for
return trained agent

References